



PYTHON SUPER-DUPER MASTER TEST • 50 QUESTIONS

Complete coverage: Data Types & Methods • Operators • Control Flow • Functions • Error & Exception Handling • Iterators & Generators • Modules • Packages • Libraries & Frameworks

Student Name	<u>Devika</u>	Marks	<u> </u> / 100
Date	<u>Sep11-2026</u>	Time	120 min
Batch / Section	<u> </u>	Instructor	<u> </u>

□ INSTRUCTIONS

Answer all 50 questions. MCQs: choose one best answer. Output questions: show exact output. Coding questions: write complete Python 3.x code in the provided space. For debugging, identify the error and correct it. Clean indentation and readable code are expected.

PART A — MULTIPLE CHOICE

01. Which expression returns a float? A) 10//3 B) 10/3 C) int(10/3) D) 10%3
02. Which type is immutable? A) list B) set C) tuple D) dict
03. What is `(3+4j).conjugate()`? #sign change A) 3-4j B) 4-3j C) -3+4j D) 3+4j
04. What does `bool(0)` return? A) True B) False C) 0 D) Error
05. Which string method removes whitespace from both ends? A) strip() B) split() C) trim() D) clean()
06. Which string method combines an iterable of strings? A) split() B) join() C) merge() D) concat()
07. Which list method adds one item at the end? A) extend() B) append() C) insert() D) add()
08. Which list method adds multiple items from an iterable? A) append() B) extend() C) insert() D) update()
09. Which set method removes an item without raising KeyError if missing? A) remove() B) discard() C) pop() D) delete() *discard() -> no error if item is missing *remove() -> KeyError if item is missing
10. Which dictionary method returns key-value pairs? A) keys() B) values() C) items() D) pairs()

11. Which tuple method returns the number of occurrences?

A) **count()** B) index() C) occurrences() D) find()

12. Which function converts a value to an integer?

A) str() B) **int()** C) integer() D) number()

13. Which operator performs exponentiation?

A) ^ B) ****** C) // D) ^^

14. What is the result of `17 % 5`?

A) **2** B) 3 C) 4 D) 5

15. Which operator checks object identity?

A) == B) **is** C) in D) =

16. Which logical operator is true only when both operands are true?

A) or B) **and** C) not D) xor

17. Which keyword skips the current loop iteration?

A) break B) **continue** C) pass D) skip

18. Which keyword exits a loop immediately?
A) stop B) continue C) break D) exitloop

19. Which statement creates a conditional expression? *#also called ternary expression*
A) x if condition else y B) if x then y C) choose(x,y) D) condition ? x : y

20. Which parameter syntax collects extra positional arguments?
A) **args B) *args C) &args; D) args[]

PART B — OUTPUT & DEBUGGING

21. Predict the output:

```
x=[1,2,3]; x.append(4); x.pop(1); print(x)
```

x.append(4) -> [1,2,3,4]

x.pop(1) -----> [1,3,4]#####pop(1) index 1

22. Predict the output:

```
print('Python'.upper().replace('PY','MY'))
```

PYTHON

MYTHON

23. Predict the output:

```
a={1,2,3}; b={3,4}; print(a.intersection(b))
```

{3}

24. Predict the output:

```
d={'a':10}; print(d.get('b',99))
```

99---because b is not exist so default value 99 prints

25. Identify the exception raised by `10 / 0`.

ZeroDivisionError

26. Identify the exception raised by `int('abc')`.

`ValueError: invalid literal for int()`

27. Fix the syntax/logic issue: `if x = 10:`

`if x == 10:`

28. What happens here?

```
it=iter([10,20])
print(next(it))
print(next(it))
print(next(it))
```

after 10 20 there is no other value so stop the iteration.

PART C-ERROR & EXCEPTION HANDLING

29. Differentiate Syntax Error, Runtime Error and Logical Error. Give one Python example of each.

Syntax Error → mistake in code syntax.

Example: `if x == 10`

Runtime Error → error while program is running.

Example: `10 / 0`

Logical Error → program runs but gives wrong output.

`age = 40`

```
if age > 60:
    print("Adult")
else:
    print("Child")
```

o/p:40 but 40 is not child age

30. Explain Error vs Exception. Why is exception handling important in automation/programs?

Error → problem in program.

`if x = 10`

Exception → runtime error that can be handled using `try/except`.

Exception handling prevents the program from crashing.

Ex: `a=10`

`b=0`

`print(10 / 0)`

31. Write one example demonstrating try + except + else + finally. Explain when each block executes.

`try` → risky code

`except` → handles error

`else` → runs if no error
`finally` → always runs

`try:`

```
    print(10 / 2)
```

`except ZeroDivisionError:`

```
    print("Error")
```

`else:`

```
    print("Success")
```

`finally:`

```
    print("Done")
```

32. Write code handling multiple exceptions, for example `ValueError` and `ZeroDivisionError`, with separate `except` blocks.

`try:`

```
    n = int(input())
```

```
    print(10 / n)
```

`except ValueError:`

```
    print("Invalid number")
```

`except ZeroDivisionError:`

```
    print("Cannot divide by zero")
```

33. Use `raise` to reject an invalid age. Then create and handle a custom `AgeError`.

```
age = -1

if age < 0:

    raise ValueError("Invalid age")
```

custom Exception:

```
class AgeError(Exception):

    age = -5

try:

    if age < 0:

        raise AgeError("Age cannot be negative")

    print("Valid age")

except AgeError as e:

    print(e)
```

34. Explain the Python Exception Hierarchy. Include BaseException, Exception and at least four common subclasses.

BaseException

Exception

ValueError

TypeError

KeyError

ZeroDivisionError

PART D — ITERATORS & GENERATORS

35. Explain iterable vs iterator. Demonstrate iter() and next() with a list.

Iterable → An **iterable** is something you can loop through. A list is an iterable because we can use a `for` loop on it.

Iterator → gets values one by one using `next()`.

```
a = [10, 20]
it = iter(a)

print(next(it))
o/p: 10
```

`iter(a)` creates an iterator.

`next(it)` gets the first value, which is 10.

36. Create a custom iterator class Countdown implementing `iter()` and `next()`.

37. Write a generator function `even_numbers(n)` using `yield`.

```
def even_numbers(n):
```

```
    for i in range(n + 1):
```

```
        if i % 2 == 0:
```

```
            yield i
```

```
for num in even_numbers(10):
```

```
    print(num)
```

38. Write a generator expression that produces squares from 1 to 20.

```
def squares():
```

```
    for i in range(1, 21):
```

```
        yield i * i
```

```
for value in squares():
```

```
    print(value)
```

39. Explain StopIteration. When does it occur and how does a for loop handle it?

```
nums = iter([10, 20])

print(next(nums))  # 10
print(next(nums))  # 20
print(next(nums))  # StopIteration
```

A for loop handles StopIteration automatically. When there are no more values, it simply stops the loop.

```
for num in [10, 20]:
    print(num)
```

40. Compare Generator vs List in terms of memory, execution and reuse. Give an example.

List: values are created at once and can be reused.

```
numbers = [10, 20, 30]

print(numbers[0])
print(numbers[1])
print(numbers[2])

print(numbers[0]) # works again we can reuse
```

Generator: creates one value at a time and is usually used once.

```
# Generator
def numbers():
    yield 10
    yield 20
    yield 30
```

```
g = numbers()
```

```
print(next(g))
print(next(g))
print(next(g))
```

```
# Reuse
print(next(g)) # StopIteration-we cannot reuse
```

PART E — MODULES, PACKAGES & LIBRARIES

41. Create a module `calculator.py` with `add()`, `subtract()`, `multiply()` and show import usage.

Create a file named `calculator.py`:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b
```

Then in another file, for example `sample.py`:

```
import calculator  
  
print(calculator.add(10, 5))  
print(calculator.subtract(10, 5))  
print(calculator.multiply(10, 5))
```

42. Demonstrate `import`, `from ... import` and an alias using ``as``.

Using the same `calculator.py`:

```
# 1. import  
import calculator  
  
print(calculator.add(10, 5))  
# 2. from ... import  
from calculator import add  
  
print(add(10, 5))  
# 3. alias using as  
import calculator as cal  
  
print(cal.multiply(10, 5))
```

43. Explain `name` and `__main__`. Write a correct `if __name__ == '__main__':` example.

44. Design a package tree with `__init__.py`, nested packages, utilities and tests. Show valid imports.

PART F — FUNCTIONS & INTEGRATED CHALLENGES

45. Explain Module vs Package vs Library vs Framework with one Python example for each.

app

├── `__init__.py`

├── `calculator.py`

└── `utilities`

 ├── `__init__.py`

 └── `helper.py`

46. Write a function using default arguments, positional arguments, keyword arguments, `*args` and `**kwargs`.

```
def payment(account, amount=100, *account_types, **details):
```

```
    print(account)
```

```
    print(amount)
```

```
    print(account_types)
```

```
    print(details)
```

```
payment("A101", 500, "Savings", "Checking", city="Austin")
```

amount=100 → default arg., If you do not give a value, it uses 100.
*account_types → *args..... It accepts multiple extra positional values and stores them as a tuple. So here savings and checkings multiple values.
**details → **kwargs..... It accepts multiple extra key=value arguments and stores them as a dictionary.

47. Write a recursive function to calculate factorial and explain the base case.

48. Write a program using if/elif/else, nested if, for, while, range(), break, continue and pass in a meaningful mini problem.

```
marks = [85, 40, 72, 25, 90]

# for, range, if/elif/else, nested if
for i in range(len(marks)):
    mark = marks[i]

    if mark >= 75:
        print("Distinction")

        if mark >= 90:
            print("Excellent")

    elif mark >= 40:
        print("Pass")

    else:
        print("Fail")

# while, break, continue, pass
count = 0

while count < 5:
    count += 1

    if count == 2:
        continue

    if count == 4:
        break

    if count == 3:
        pass

    print(count)
```

•

49. Build a reusable automation-style utility module that validates input, catches exceptions and returns a clean result.

```
# utility.py

def divide(a, b):

    try:

        a = int(a)

        b = int(b)

        return a / b

    except ValueError:

        return "Please enter valid numbers"

    except ZeroDivisionError:

        return "Cannot divide by zero"

####

from utility import divide

print(divide("10", "2"))
print(divide("10", "0"))
print(divide("abc", "2"))
```

Output:

```
Result: 5.0
Cannot divide by zero
Please enter valid numbers
```

50. □ CAPSTONE: Build a Student Result Analyzer. Input marks, validate bad input, calculate total/average/grade, use functions, a custom exception, a generator, a reusable module and a package-style structure. Add a clean report.

